C语言程序设计课堂练习题解答

练习题一

第1题 — C语言合法标识符

> 下列选项中，哪些是合法的C语言标识符？
> ①print ②FOR ③&a ④*00* ⑤*int* ⑥default

合法的有：①②④⑤

选项	合法?	说明
`print`	✅	不是关键字（`printf` 才是库函数名，但标识符合法）
`FOR`	✅	C区分大小写，关键字是 `for`，`FOR` 是合法标识符
`&a`	❌	含特殊字符 `&`，不能出现在标识符中
`_00`	✅	以下划线开头，后跟数字，合法
`int_`	✅	虽然含 `int`，但整体不是关键字，合法
`default`	❌	是C语言关键字

> **规则**：标识符由字母、数字、下划线组成，且不能以数字开头，不能是关键字。

第2题 — 表示32768

> 已知int类型数据在内存中存储长度为2字节，写出能正确表示32768的数据类型。

答案： `long x = 32768;` **（或** `32768L` **）**

2字节 `int` 的范围是 **-32768 ~ 32767**，无法存储32768。需要使用 `long` 类型（4字节）。

```
long x = 32768;   // 正确
// 或
long x = 32768L;  // 显式标注长整型常量
```

第3题 — 表达式求值

> 已知定义 `int a=3, b=4, c=5;`，求下列表达式的值。

表达式	值	解析
`a&&b`	**1**	3非零且4非零 → 逻辑真 → 1
`a<=b`	**1**	3 ≤ 4 为真 → 1

表达式	值	解析
`a||b&&c`	**1**	`&&` 优先级高于 `||`：先算 `b&&c` =1，再算 `a||1` =1
`!(!c||1)`	**0**	`!c` =0 → `0||1` =1 → `!1` =0

第4题 — 整数除法陷阱

> 已知定义 `int a=3, b=4, c;`，执行 `c=1/2*(a+b);` 后，c的值为？

答案：c = 0

```
1 / 2 = 0          // 整数除法，截断小数部分
0 * (3 + 4) = 0    // 0乘以任何数都是0
```

> **易错点**：`1/2` 在C中是整数除法，结果为0，而不是0.5。若需要浮点结果应写 `1.0/2*(a+b)`。

第5题 — 合法表达式

> 已知定义 `int a=3, b=4;`，哪些表达式合法？

表达式	合法?	说明
`a+b=7`	❌	`a+b` 不是左值，不能出现在赋值号左边
`a=|b|`	❌	C语言没有 `|x|` 取绝对值运算，应用 `abs(b)`
`a=b=0`	✅	连续赋值，从右到左：`b=0`，`a=0`
`(a++)++`	❌	`a++` 的结果是右值（临时值），不能再 `++`
`Double(a)/10`	❌	C没有 `Double()` 函数，强制转换应写 `(double)a/10`
`++a, b=a--`	✅	逗号表达式：先 `++a` 使a=4，再 `b=a--` 使b=4、a=3

合法的：③⑥

第6题 — 条件判断的不同实现

> 当x≥0时y=1，否则y=-1，下列实现正确的是？

方案	正确?	说明
① `if(x>=0) y=1; else y=-1;`	✅	标准if-else写法
② `y=x>=0?1:-1;`	✅	条件运算符，简洁正确
③ `switch(){case x>=0:...}`	❌	`case` 后必须是**常量表达式**，`x>=0` 是变量表达式
④ `switch(x-abs(x)){case 0:y=1;break;default:y=-1;}`	✅	x≥0时 `x-abs(x)=0`；x<0时 `x-abs(x)=2x≠0`

正确的：①②④

第7题 — if-else配对（悬挂else问题）

```c
int a=1, b=0;
if(!a) b++;
else if(a==0) if(a) b+=2;
else b+=3;
printf("%d\n", b);
```

答案：输出 `0`

分析控制流：

1. `a=1`， `!a` =0 → `if(!a)` 为**假**，进入 `else`
2. `else` 中： `if(a==0)` → `a=1`，条件为**假**
3. 由于 `if(a==0)` 为假，其内部的 `if(a) b+=2; else b+=3;` 整体被跳过
4. `b` 始终为0

> **关键**： `else` 与**最近的未匹配** `if` 配对。这里外层else和if(!a)配对，内层else和if(a)配对。但由于if(a==0)就已经为假，内部代码不执行。

第8题 — 去掉最高位和最低位

> 已知 `long x, y;`，x是n位十进制数(4<n<10)，去掉x最高位和最低位，将剩余数赋给y。

答案：

```c
y = x % (long)pow(10, n-1) / 10;
```

原理：

- `x % (long)pow(10, n-1)` → 去掉最高位（取模去掉最高位的权值）
- `/ 10` → 去掉最低位（整数除以10丢弃个位）

验证：x=12345, n=5

- `12345 % 10000 = 2345` （去掉了最高位1）
- `2345 / 10 = 234` （去掉了最低位5）
- y = 234 ✓

第9题 — 悬挂else + if(0)

```c
int a=0, b=1, d=10;
if(a)
    if(b)
        d=20;
    else
        d=30;
printf("%d\n", d);
```

答案：输出 `10`

- `a=0`，`if(a)` 为**假**
- 整个 `if(b)...else...` 作为 `if(a)` 的真分支，被完全**跳过**
- `d` 保持初始值10

> **注意**：`else` 和内层 `if(b)` 配对，而不是和外层 `if(a)` 配对。但这不影响结果——因为外层 `if(a)` 已经为假，整个内部块不执行。

第10题 — while + switch 综合

```c
int k=5, n=0;
while(k>0) {
    switch(k) {
        case 1:
        case 3: n+=1; k--; break;
        default: n=0; k--;
        case 2:
        case 4: n+=2; k--; break;
    }
}
printf("%3d", n);
```

答案：输出 `6`（即6，前面有两个空格）

逐步追踪：

轮次	k进入	匹配	执行过程	k离开	n
1	5	default	n=0, k=4; **穿透**到case 2/4 → n=2, k=3; break	3	2
2	3	case 3	n=3, k=2; break	2	3
3	2	case 2	n=5, k=1; break	1	5
4	1	case 1→穿透case 3	n=6, k=0; break	0	6
5	0	退出while	—	—	6

> **关键**：`default` 后面没有 `break`，会穿透(fall through)到下面的 `case 2` / `case 4`。

第11题 — 双重for + break

```c
int i, j, m=1;
for(i=1; i<3; i++) {
    for(j=3; j>0; j--) {
        if((i*j) > 3) break;
        m = i*j;
    }
}
printf("m=%d\n", m);
```

答案：输出 `m=1`

i	j	i*j	>3?	动作
1	3	3	否	m=3
1	2	2	否	m=2
1	1	1	否	m=1
2	3	6	**是**	break（退出内层循环）

- `i=2` 时，`j=3` 就直接 `break` 了，m没有被更新
- 最后m保持为1（i=1循环最后一次赋的值）

第12题 — for循环中修改循环变量

```c
int a=1, b=2;
for(a=1; a<8; a++) {
    b += a;
    a += 2;
}
printf("%d,%d\n", a, b);
```

答案：输出 `10,14`

进入循环时a	b += a	a += 2	循环自增a++	退出时a
1	b=2+1=3	a=3	a=4	4
4	b=3+4=7	a=6	a=7	7
7	b=7+7=14	a=9	a=10	10
10	10<8为假，退出	—	—	10

> **注意**：循环体内 `a+=2` 和循环自增 `a++` 都会修改a，每轮a实际增加3。

练习题二

第1题 — 合法的C语言常量

> A) 1,234　　B) '\123'　　C) 123　　D) "\x7G"

答案：B

选项	合法?	说明
A) `1,234`	❌	逗号不是数字常量的一部分（C不支持千位分隔符）
B) `'\123'`	✅	八进制转义字符，`\123` = 1×64+2×8+3 = 83 = `'S'`
C) `123`	✅	合法整数常量（若为单选则B更典型）
D) `"\x7G"`	❌	`G` 不是十六进制数字(0-9, A-F)，`\x7G` 非法

第2题 — 合法的实型常量

> A) .1e0　　B) 3.0e0.2　　C) E9　　D) 9.12E

答案：A

选项	合法?	说明
A) `.1e0`	✅	等于$0.1\times10^0 = 0.1$，合法
B) `3.0e0.2`	❌	指数部分必须是**整数**，0.2不合法
C) `E9`	❌	E前面必须有数字部分
D) `9.12E`	❌	E后面必须有指数

第3题 — 错误的赋值表达式

> int a=3, b=2, c=1; 哪个赋值表达式是**错误**的?
> A) a=(b=4)=3　　B) a=b=c+1　　C) a=(b=4)+c　　D) a=1+(b=c=4)

答案：A

- `(b=4)` 的结果是**右值**（值4），不能再被赋值
- `(b=4)=3` 相当于 `4=3`，左边不是左值，**非法**

其他三项都是合法的连续赋值或复合赋值。

第4题 — switch穿透 + 循环输入

```c
int s;
scanf("%d", &s);
while(s > 0) {
    switch(s) {
        case 1: printf("%d", s+5);        // 无break，穿透
        case 2: printf("%d", s+4); break;
        case 3: printf("%d", s+3);        // 无break，穿透
        default: printf("%d", s+1); break;
    }
    scanf("%d", &s);
}
```

输入：`1 2 3 4 5 0`

答案：A) 6566456

输入s	匹配	输出
1	case 1 → 穿透case 2	`6` (1+5), `5` (1+4) → `"65"`
2	case 2	`6` (2+4) → `"6"`
3	case 3 → 穿透default	`6` (3+3), `4` (3+1) → `"64"`

输入s	匹配	输出
4	default	`5` (4+1) → `"5"`
5	default	`6` (5+1) → `"6"`
0	退出while	—

总输出拼接：`65` + `6` + `64` + `5` + `6` = `6566456`

第5题 — 字符串中的\0截断

```c
char s[] = "012xy\08s34f4w2";
int i, n = 0;
for(i=0; s[i]!=0; i++)
    if(s[i]>='0' && s[i]<='9') n++;
printf("%d\n", n);
```

答案：输出 `3`

字符串在内存中的存储：`'0','1','2','x','y','\0','8','s','3','4',...`

- `\0` 是八进制转义，值为0（就是空字符）
- `\08` 中 `8` 不是合法八进制数字，所以 `\0` 是一个独立的转义字符
- 循环遇到 `s[5]='\0'` 时终止
- 只统计了前5个字符中的数字：`'0'`，`'1'`，`'2'` → n=3

第6题 — 赋值作为循环条件

```c
for(i=0, k=-1; k=1; k++) printf("*****\n");
```

答案：D) 无限循环（死循环）

- 条件是 `k=1`（赋值），不是 `k==1`（比较）
- `k=1` 每次都将k设为1，表达式的值为1（真）
- 即使 `k++` 使k变为2，下次判断时又被赋值为1 → 永远为真

第7题 — 字符交替大小写输出

```c
char b, c; int i;
b = 'a'; c = 'A';
for(i=0; i<6; i++) {
    if(i%2) putchar(i+b);
    else putchar(i+c);
}
```

答案：B) AbCdEf

i	i%2	输出字符	计算
0	0(else)	`'A'`	0+'A'=65='A'
1	1(if)	`'b'`	1+'a'=98='b'
2	0(else)	`'C'`	2+'A'=67='C'
3	1(if)	`'d'`	3+'a'=100='d'
4	0(else)	`'E'`	4+'A'=69='E'
5	1(if)	`'f'`	5+'a'=102='f'

第8题 — 给double数组元素读入数据

> `double x[10], *p=x;` 正确读入x[6]的语句是?

答案：C) `scanf("%lf", p+6);`

选项	正确?	说明
A) `scanf("%f", &x[6])`	❌	`%f` 用于 `float`， `double` 必须用 `%lf`
B) `scanf("%lf", *(x+6))`	❌	`*(x+6)` 是x[6]的**值**，不是地址
C) `scanf("%lf", p+6)`	✅	`p+6` 等价于 `&x[6]`，格式符 `%lf` 正确
D) `scanf("%lf", p[6])`	❌	`p[6]` 等价于 `*(p+6)`，是值不是地址

第9题 — 按ASCII奇偶筛选字符

```c
void fun(char *s) {
    while(*s) {
        if(*s % 2) printf("%c", *s);
        s++;
    }
}
// 调用: fun("BYTE");
```

答案：D) YE

字符	ASCII值	%2	是否输出
B	66	0	否
Y	89	1	**是**
T	84	0	否
E	69	1	**是**

输出： `YE`

第10题 — 指针类型匹配

> `char s[3][10], (*k)[3], *p;` 正确的赋值是？

答案：C) `p=s[0];`

选项	正确?	说明
A) `p=s`	❌	`s` 类型是 `char(*)[10]`（指向含10个char的数组），`p` 是 `char*`，类型不匹配
B) `p=k`	❌	`k` 类型是 `char(*)[3]`，`p` 是 `char*`，类型不匹配
C) `p=s[0]`	✅	`s[0]` 退化为 `char*`，与 `p` 类型一致
D) `k=s`	❌	`s` 是 `char(*)[10]`，`k` 是 `char(*)[3]`，数组大小不同

第11题 — scanf给数组元素赋值（含i自增）

```c
void fun(int x[N]) {
    int i = 0;
    while(i < N) scanf("%d", _______);
}
```

答案：C) `x+(i++)`

选项	说明
A) `x+i`	地址正确，但 `i` 永远不变 → 死循环
B) `&x[i+1]`	跳过x[0]，且 `i` 不递增 → 死循环
C) `x+(i++)`	✅ 地址正确，`i++` 后自增保证遍历
D) `&x[++i]`	从x[1]开始，跳过x[0]，且最后访问越界

第12题 — 结构体赋值

> `struct {char mark[12]; int num1; double num2;} t1, t2;` 正确的赋值语句？

正确的：A、B、D

选项	正确?	说明
A) `t1=t2`	✅	结构体可以整体赋值
B) `t2.num1=t1.num1`	✅	int成员赋值
C) `t2.mark=t1.mark`	❌	**数组名是常量地址，不能赋值**。应用 `strcpy(t2.mark, t1.mark)`
D) `t2.num2=t1.num2`	✅	double成员赋值

第13题 — 结构体指针访问

```c
struct ord { int x, y; } dt[2] = {1,2,3,4};
void main() {
    struct ord *p = dt;
    printf("%d,", ++(p->x));
    printf("%d\n", ++(p->y));
}
```

答案：D) 2,3

- `dt[0] = {x=1, y=2}`，`dt[1] = {x=3, y=4}`
- `p` 指向 `dt[0]`
- `++(p->x)`：先取 `p->x` =1，再 `++` → **2**，输出 `2,`
- `++(p->y)`：先取 `p->y` =2，再 `++` → **3**，输出 `3`

练习题三

第1题 — 合法的用户标识符

> A) void, define, WORD　B) a3_3, _123, Car　C) For, -abc, IF Case　D) 2a, DO, sizeof

答案：B

选项	说明
A)	`void` 是关键字 ❌
B)	`a3_3`、`_123`、`Car` 都是合法标识符 ✅
C)	`-abc` 含减号 ❌
D)	`2a` 以数字开头 ❌，`sizeof` 是关键字 ❌

第2题 — `char *line[5]` 的含义

答案：A

`char *line[5]` 定义 `line` 是一个**包含5个元素的数组**，每个元素是 `char*` 类型的指针。

- 优先级：`[]` > `*`，所以 `line` 先与 `[5]` 结合 → 数组
- 每个数组元素是 `char*` → 指针数组

第3题 — 正确的定义语句

> A) int a=b=0;　B) char A=65+1, b='b';　C) float a=1, *b=&a, *c=&b;　D) double a=0;b=1.1;

答案：B

选项	正确?	说明
A) `int a=b=0`	❌	`b` 未事先声明，不能在定义语句中连续赋值
B) `char A=65+1, b='b'`	✅	A=66='B'，b='b'，合法
C) `float a=1, *b=&a, *c=&b`	❌	`&b` 是 `float**` 类型，不能赋给 `float*` 类型的c
D) `double a=0;b=1.1`	❌	分号断开后 `b` 无类型声明

第4题 — 被15整除的数之和

```c
int i, s = 1;
for(i=1; i<50; i++)
    if(!(i%5) && !(i%3)) s += i;
printf("%d\n", s);
```

答案：输出 `91`

条件 `!(i%5) && !(i%3)` 表示 i 同时能被5和3整除，即能被**15**整除。

1~49中能被15整除的数：**15, 30, 45**

```
s = 1 + 15 + 30 + 45 = 91
```

第5题 — 嵌套switch穿透

```c
int x=1, y=0, a=0, b=0;
switch(x) {
    case 1:
        switch(y) {
            case 0: a++; break;     // 内层break
            case 1: b++; break;
        }
    case 2: a++; b++; break;      // 外层case 2
    case 3: a++; b++;
}
printf("a=%d,b=%d\n", a, b);
```

答案：输出 `a=2,b=1`

执行流程：

1. `x=1` → 进入外层 `case 1`
2. 内层 `switch(y)`： `y=0` → `case 0`： `a++` → a=1； `break` 跳出**内层switch**
3. 外层 `case 1` 之后没有 `break` → **穿透**到 `case 2`
4. `case 2`： `a++` → a=2； `b++` → b=1； `break` 跳出外层switch

第6题 — continue跳过偶数

```c
int i, a=0, b=0;
for(i=1; i<10; i++) {
    if(i%2 == 0) {
        a++;
        continue;   // 跳过b++
    }
    b++;
}
printf("a=%d,b=%d", a, b);
```

答案：B) a=4, b=5

- i = 1~9，共9次循环
- 偶数（2,4,6,8）：a++，`continue` 跳过b++ → a = **4**
- 奇数（1,3,5,7,9）：执行b++ → b = **5**

第7题 — 指针偏移访问数组

```c
int a[10] = {1,2,3,4,5,6,7,8,9,10}, *p = a;
printf("%d\n", *(p+2));
```

答案：输出 `3`

- `p` 指向 `a[0]`
- `p+2` 指向 `a[2]`
- `*(p+2)` = `a[2]` = **3**

第8题 — 二维数组定义

> 正确的定义是?

选项	正确?	说明
A) `int x[][3]={{0},{1},{1,2,3}}`	✅	第一维可省略，编译器自动推断为3
B) `int x[4][3]={...}`	✅	完整指定，4行3列
C) `int x[4][]={...}`	❌	**第二维不能省略**（编译器需要知道列数来计算地址）
D) `int x[][3]={1,2,3,4}`	✅	按顺序填充，自动推断第一维为2（第二行不足的补0）

错误的是C

第9题 — 字符替换函数

```c
void ss(char *s, char t) {
    while(*s) {
        if(*s == t) *s = t - 'a' + 'A';
        s++;
    }
}
// 调用: ss("abcddfefdbd", 'd');
```

答案：B) abcDDfefDbD

- `t='d'`，替换值 = `'d' - 'a' + 'A'` = `'D'`（小写转大写）
- 函数将字符串中所有 `'d'` 替换为 `'D'`

```
a b c d d f e f d b d
        ↓ ↓       ↓   ↓
a b c D D f e f D b D
```

第10题 — 给字符数组输入

> `char s[10];` 给s输入5个字符，正确的是?

答案：A 和 C 都正确

选项	正确?	说明
A) `gets(&s[0])`	✅	`&s[0]` 等价于 `s`，正确
B) `scanf("%s", s+1)`	❌	从s[1]开始存储，不是从s[0]开始
C) `gets(s)`	✅	标准用法
D) `scanf("%s", s[1])`	❌	`s[1]` 是 `char` 值，不是地址

第11题 — 字符串末尾的\0

```c
char str[] = "ABC", *p = str;
printf("%d\n", *(p+3));
```

答案：B) 0

- `str` = `{'A', 'B', 'C', '\0'}`
- `*(p+3)` = `str[3]` = `'\0'` = **0**

> 字符串末尾自动添加 `'\0'`（ASCII值为0）。用 `%d` 输出就是**0**。

第12题 — 函数指针参数传递

```c
void func(int *a, int b[]) {
    b[0] = *a + 6;
}
void main() {
    int a, b[5];
    a = 0; b[0] = 3;
    func(&a, b);
    printf("%d\n", b[0]);
}
```

答案：输出 `6`

- `func` 接收：`a` 指向main中的a（值为0），`b` 指向main中的b数组
- `b[0] = *a + 6 = 0 + 6 = 6`
- 由于数组通过指针传递，修改直接作用于原数组
- 输出 **6**

> 注意：原来 `b[0]=3` 的值被覆盖了，最终结果取决于 `*a + 6` 而非原来的 `b[0]`。